

Original

MEX (minimal excluded) of a sequence

Given an array A of size N . You have to find the minimal non-negative element that is not present in the array. That number is commonly called the **MEX** (minimal excluded).

$$\begin{aligned} \text{mex}(\{0, 1, 2, 4, 5\}) &= 3 \\ \text{mex}(\{0, 1, 2, 3, 4\}) &= 5 \\ \text{mex}(\{1, 2, 3, 4, 5\}) &= 0 \end{aligned}$$

Notice, that the MEX of an array of size N can never be bigger than N itself.

The easiest approach is to create a set of all elements in the array A , so that we can quickly check if a number is part of the array or not. Then we can check all numbers from 0 to N , if the current number is not present in the set, return it.

Implementation

The following algorithm runs in $O(N \log N)$ time.

```
int mex(vector<int> const& A) {
    set<int> b(A.begin(), A.end());

    int result = 0;
    while (b.count(result))
        ++result;
    return result;
}
```

If an algorithm requires a $O(N)$ MEX computation, it is possible by using a boolean vector instead of a set. Notice, that the array needs to be as big as the biggest possible array size.

```
int mex(vector<int> const& A) {
    static bool used[MAX_N+1] = { 0 };

    // mark the given numbers
    for (int x : A) {
        if (x <= MAX_N)
            used[x] = true;
    }

    // find the mex
    int result = 0;
    while (used[result])
        ++result;
    return result;
}
```